

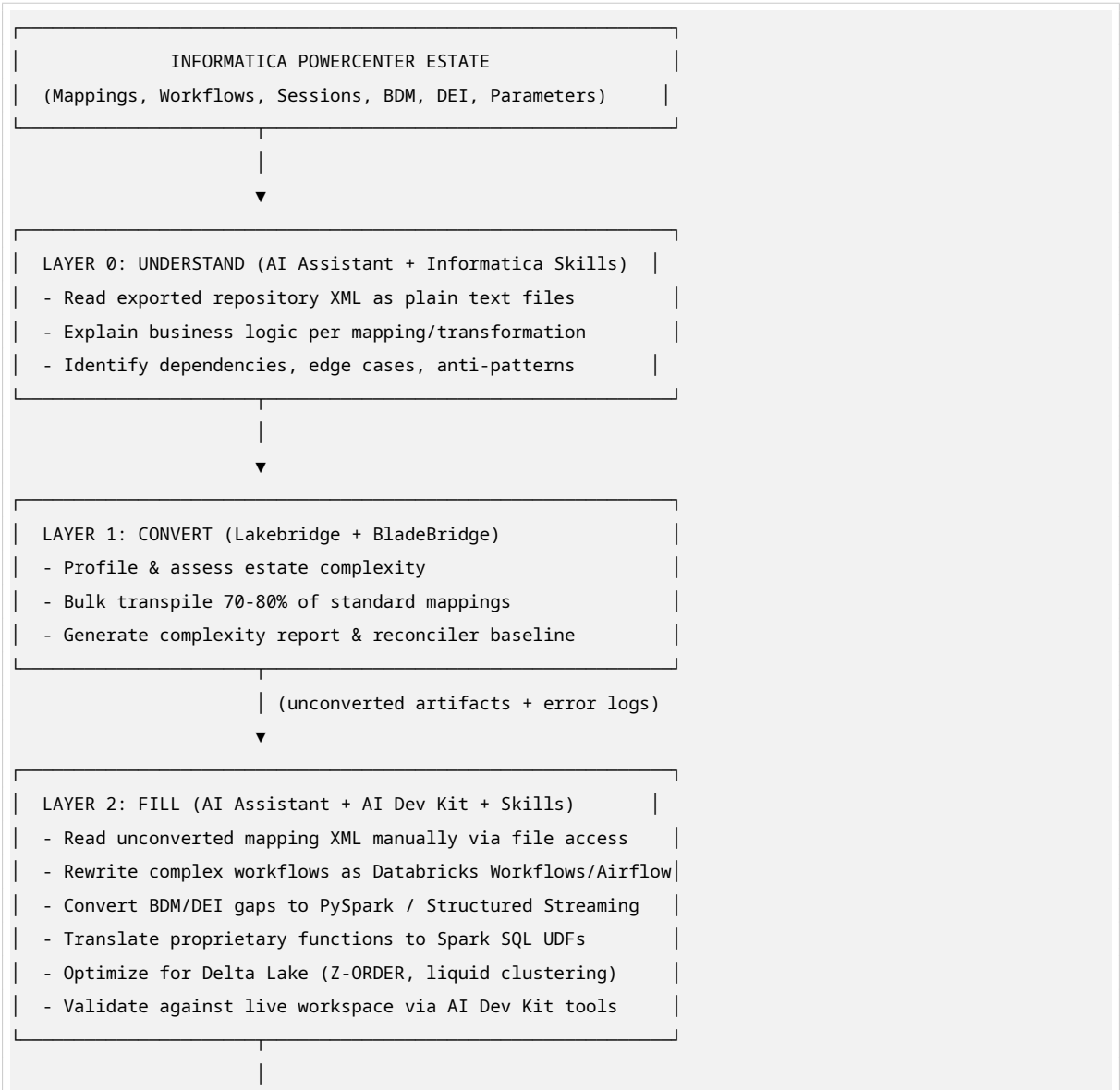
Informatica to Databricks Migration Plan

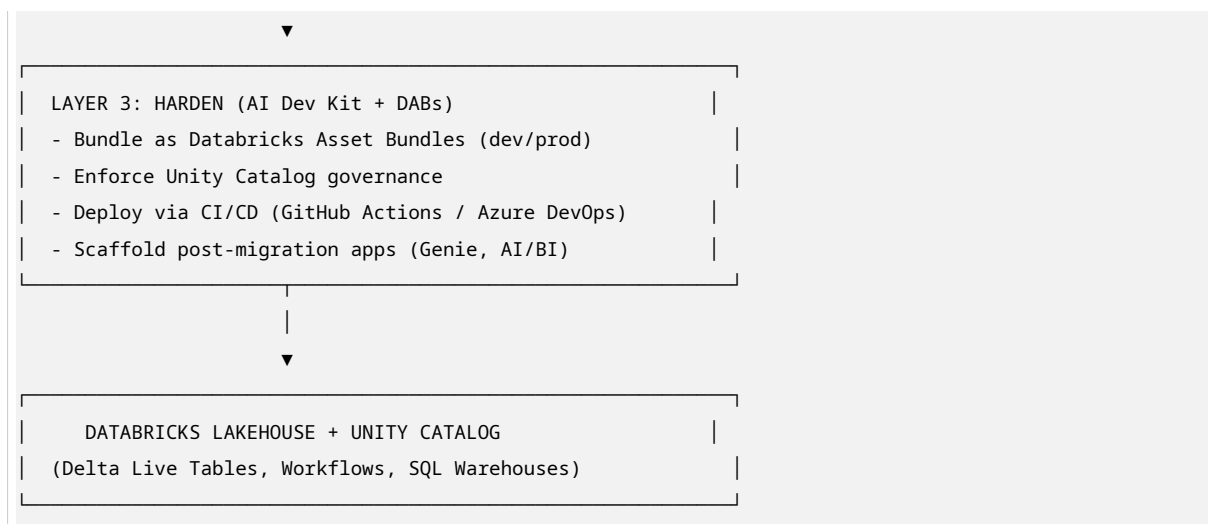
Project: Informatica PowerCenter → Databricks Lakehouse Migration
Date: 2026-05-07
Stack: Lakebridge · AI Coding Assistant · Databricks AI Dev Kit · Custom Informatica Skills

1. Executive Summary

This plan outlines a hybrid, AI-augmented migration from Informatica PowerCenter to the Databricks Data Intelligence Platform. The approach combines **Lakebridge** (bulk transpilation) with an **AI Coding Assistant** (intelligent gap-filling) powered by a custom **Informatica skill tree** and the **Databricks AI Dev Kit** (workspace-aware deployment). The AI coding assistant reads exported Informatica repository XML as plain text files and interprets them using the skills—no additional infrastructure required.

2. Architecture Overview





3. Phase-by-Phase Execution

Phase 0: Pre-Migration Preparation

Task	Owner	Output
Export Informatica repository to XML	Informatica Admin	.xml repository dump
Download official PDF documentation	Migration Team	Transformation Guide, Workflow Guide, Designer Guide, XML Guide from Informatica Documentation
Install Databricks AI Dev Kit	AI/Dev Team	Workspace integration tools + skills in project directory
Inventory all workflows & classify complexity	Migration Team	Complexity matrix (Low / Medium / Complex / Very Complex)

Key Decision: Confirm scope—PowerCenter only, or does BDM/DEI need separate tooling (LeapLogic/ Kanerika FLIP)?

Phase 1: Assessment & Bulk Conversion

Task	Tool	Details
Profile estate	Lakebridge	Run <code>lakebridge analyze</code> against exported PowerCenter XML files
Generate complexity report	Lakebridge	Identify proprietary functions, workflow depth, BDM footprint
Bulk transpile straightforward mappings	Lakebridge + BladeBridge	Target: PySpark notebooks / Spark SQL scripts
Establish reconciler baseline	Lakebridge	Schema + row-level + column-level reconciliation rules

Table 2 – continued

Task	Tool	Details
Document gaps	Manual review	Log unconverted mappings, complex workflows, BDM assets

Deliverable: Converted notebook baseline + gap registry.

Phase 2: Intelligent Gap-Filling

Task	Tool	Details
Read unconverted Informatica XML	AI assistant (file access)	Open exported XML files directly; use Informatica skills to interpret structure
Explain legacy logic in business terms	AI assistant + Informatica skills	“What does <code>m_Customer_Load</code> do?” → natural language + technical breakdown
Rewrite complex mappings	AI assistant + AI Dev Kit	Unconnected lookups → broadcast joins; blocking joiners → sorted-merge; SCD Type 2 → DLT
Reconstruct workflow orchestration	AI assistant	Event waits → Databricks Workflows conditions; worklets → task groups; shell tasks → Jobs/DBUtils
Translate proprietary functions	AI assistant + function skills	IIF → CASE WHEN; ISNULL → COALESCE; TO_DATE with Informatica format strings → Spark <code>to_date</code>
Live validation	AI Dev Kit	Execute generated SQL against SQL warehouse; create test jobs; inspect logs
Data reconciliation	Lakebridge reconciler + custom tests	Side-by-side row counts, hash diffs, aggregate parity

Deliverable: Fully converted pipeline set + validation report.

Phase 3: Production Hardening & Deployment

Task	Tool	Details
Create Databricks Asset Bundles	AI Dev Kit	<code>databricks.yml</code> with dev/staging/prod targets
Apply Unity Catalog governance	AI Dev Kit + manual	Catalogs, schemas, managed tables, volume mounts, access controls
CI/CD integration	GitHub Actions / Azure DevOps	Automated bundle deployment on PR merge
Performance optimization	AI assistant + AI Dev Kit	Z-ORDER, OPTIMIZE, liquid clustering, partition tuning

Table 4 – continued

Task	Tool	Details
Parallel running (dual systems)	Operational	Keep Informatica live; run Databricks in shadow; compare outputs until parity is proven
Cutover & decommission	Operational	Domain-by-domain switchover; retire Informatica workflows incrementally

Deliverable: Production-grade Databricks estate + decommissioned Informatica domain.

4. Tool Stack

4.1 Lakebridge (Foundation Layer)

Role: Bulk transpilation and reconciliation.

What is Lakebridge? **Databricks Lakebridge** is a free, open-source toolkit developed by Databricks Labs designed to automate and accelerate migrations from legacy data warehouses and ETL platforms to Databricks SQL and the Lakehouse architecture [17][59]. It provides an end-to-end migration experience covering assessment, code conversion, and data validation—automating up to 80% of otherwise manual migration tasks [17][23].

Lakebridge originated from Databricks field engineers and is now used by over 1,000 customers and partners, with adoption growing approximately 20% month-over-month [19][63]. It is formally maintained as an open-source project under Databricks Labs, meaning it is provided without SLA guarantees but with active community support via GitHub Issues [24].

Key capabilities:

- **Analyzer:** Scans legacy codebases to assess complexity, identify dependencies, and estimate migration effort [16][57].
- **Converter / Transpiler:** Translates SQL and ETL code from 10+ source platforms into Databricks-compatible Spark SQL or PySpark notebooks [17][23].
- **Validator / Reconciler:** Compares source and target datasets to ensure data accuracy after migration [16][23].

Supported source platforms (as of early 2026) include Microsoft SQL Server, Oracle, Snowflake, Amazon Redshift, Azure Synapse Analytics, Teradata, Netezza, Hive, and others—with Informatica PowerCenter supported via the BladeBridge transpiler engine [17][20][23].

Transpiler Engines Inside Lakebridge Lakebridge contains three transpiler engines internally. When you run `lakebridge transpile --source-dialect informatica`, Lakebridge selects the appropriate engine automatically:

1. **BladeBridge** — mature, rule-based engine acquired by Databricks in Feb 2025. Best for Informatica PowerCenter and complex ETL orchestration. Used by Accenture, Capgemini, and Tredence for enterprise migrations [16][20].
2. **Morpheus** — next-generation SQL transpiler with experimental dbt support. Handles smaller dialect sets with cleaner output architecture [16].
3. **Switch** — LLM-powered transpiler for nuanced translation of complex procedural logic and edge-case syntax [16].

For your Informatica PowerCenter migration, BladeBridge is the relevant engine. It parses PowerCenter repository XML and converts mappings into PySpark notebooks or Spark SQL scripts using a configuration-driven rule engine. You do not install or manage BladeBridge separately—it ships as part of the Lakebridge package.

Where to Get It Lakebridge is distributed freely through the Databricks Labs GitHub repository and installed via the Databricks CLI.

Prerequisites:

- **Databricks workspace** (production, dev, or free trial at databricks.com/try-databricks) [60]
- **Databricks CLI** installed and authenticated (Personal Access Token or Service Principal) [60]
- **Python** 3.10.1 through 3.13.x [60]
- **Java** 11 or higher (required for the Morpheus transpiler component) [60]
- **Network access** to GitHub, Maven Central, and PyPI (or internal mirrors in restricted environments) [60]

Installation steps:

```
# 1. Authenticate with your Databricks workspace
databricks auth login

# 2. Install lakebridge core
databricks labs install lakebridge

# 3. Verify installation
databricks labs lakebridge --help

# 4. Install transpiler dependencies
databricks labs lakebridge install-transpile
```

Official resources:

- **Documentation:** <https://databrickslabs.github.io/lakebridge/> [59]
- **GitHub repository:** <https://github.com/databrickslabs/lakebridge> [24]
- **PyPI package:** `databricks-labs-lakebridge` [61]

How It Works Lakebridge operates through three integrated phases that mirror the migration lifecycle:

Phase 1: Analyze (Pre-Migration Assessment) The Analyzer performs a deep scan of the legacy environment to answer two critical questions before any code is converted: **What is the scope?** and **How complex is this?**

What it does:

- Scans SQL scripts, stored procedures, views, and ETL metadata line-by-line [^58^]
- Identifies syntax patterns, complexity tiers, and unsupported constructs
- Catalogs all objects requiring migration (tables, views, jobs, stored procedures)
- Classifies workloads into complexity buckets: **Low / Medium / Complex / Very Complex** [^17^]
- Generates a multi-tabbed Excel/JSON report with TCO estimates and risk indicators [^23^]

Usage:

```
databricks labs lakebridge analyze --source-dialect informatica --input-source /path/to/exported/
↪ sql/files/ --report-name migration_assessment
```

Output: A detailed assessment report that feeds directly into project planning and budgeting. For a real-world benchmark, one team analyzed 3,500+ SQL Server files in under 10 minutes [^23^]. Informatica PowerCenter assessment times will vary based on repository size and mapping complexity.

Phase 2: Transpile (Code Conversion) The Transpiler is the core conversion engine. Lakebridge supports three transpilers internally, selected based on source platform and migration needs [^16^]:

Transpiler	Best For	Description
BladeBridge	Mature, wide dialect support	Rule-based engine acquired by Databricks in Feb 2025. Trusted by Accenture, Capgemini, Tredence for hundreds of enterprise migrations. Handles ETL orchestration and complex SQL dialects [^16][^20].
Morpheus	Next-generation SQL	Modern transpiler with experimental dbt support. Handles smaller dialect sets but with cleaner output architecture [^16].
Switch	LLM-powered conversion	Uses Large Language Models for nuanced translation of complex procedural logic and edge-case syntax [^16].

For Informatica PowerCenter migrations, BladeBridge is the relevant engine. It parses PowerCenter repository XML and converts mappings into PySpark notebooks or Spark SQL scripts using

a configuration-driven rule engine. When unit tests fail, engineers adapt configuration files rather than rewriting code manually—enabling fast, iterative refinement [20][22].

Usage:

```
databricks labs lakebridge transpile --source-dialect informatica --input-source /path/to/
↪ informatica/export/ --output-folder /path/to/databricks/notebooks/ --error-file-path /path/to/
↪ gaps.log
```

Important practice: Run transpilation on a representative subset first, validate output quality, then tune configuration files before processing the full estate. Lakebridge can be re-run multiple times iteratively [23].

Phase 3: Reconcile (Post-Migration Validation) The Reconciler ensures that data in Databricks matches the source system exactly. It supports four validation modes [23]:

The Reconciler ensures that data in Databricks matches the source system exactly. It performs validation at three levels [16][23]:

- **Schema-level:** Compares column names, data types, and nullability between source and target
- **Row-level:** Performs hash-based comparison to identify missing or different rows
- **Column-level:** Deep-dive value comparison using join keys for cell-level mismatch analysis

Usage:

```
# Step 1: Configure connections (one-time setup)
databricks labs lakebridge configure-reconcile \
  --profile DEFAULT \
  --source-connection <source_warehouse> \
  --target-connection <target_warehouse>

# Step 2: Run reconciliation
databricks labs lakebridge reconcile \
  --reconciliation-config /path/to/config.yaml
```

Note: Reconciler currently supports direct connections to Snowflake, Oracle, and Databricks tables as sources. For Informatica, reconciliation is typically performed after data has been landed in Databricks via separate ETL, comparing legacy warehouse tables to new Delta Lake tables [23].

Limitations for Informatica Migrations While Lakebridge is powerful, be aware of these constraints specific to Informatica PowerCenter:

- **PowerCenter mappings only:** BladeBridge handles PowerCenter XML mappings. It does **not** convert Informatica BDM (Big Data Management) or DEI (Data Engineering Integration) repositories [5].
- **Workflow orchestration is partial:** Complex workflow control flow (event waits, decision tasks, worklets, shell/command tasks) often requires manual redesign in Databricks Workflows or Airflow [5][27].

- **Proprietary functions:** Some Informatica-specific functions (e.g., certain data quality transformations, CLAIRE AI features) have no direct Spark equivalent and will need manual handling.
 - **AI/GenAI features upcoming:** As of early 2026, AI-powered code conversion and a GUI for Lakebridge are on the roadmap but not yet generally available [17].
-

Summary Lakebridge is the **engine** of this migration. It does the heavy structural lifting: assessing scope, converting mappings at scale, and proving data correctness. It is free, actively maintained, and backed by a growing partner ecosystem. For your Informatica estate, it will handle the bulk of mapping transpilation, while the AI assistant and the custom skills pick up the gaps that Lakebridge cannot auto-convert.

4.2 AI Coding Assistant + Informatica Skills (Intelligence Layer)

Role: Understand legacy logic, fill gaps, optimize, validate.

How it works:

1. **Informatica Skills** (installed under `.claude/skills/informatica/`, `.windsurf/`, `.github/`, or the assistant's native configuration directory) teach the AI assistant proprietary Informatica concepts.
2. **The AI assistant reads files directly**—it opens exported XML files and interprets them using the skills.
3. **Databricks AI Dev Kit** gives the AI assistant executable actions to interact with the live workspace (run SQL, create jobs, inspect Unity Catalog).

Skill Categories:

- `informatica-transformations/` — per-transformation deep dives
- `informatica-workflows/` — orchestration logic
- `informatica-repository/` — XML schema & lineage
- `informatica-functions/` — function reference & Spark equivalents
- `informatica-to-databricks-mapping/` — the master Rosetta Stone

Key Prompt Pattern:

```
"Using the informatica-lookup skill, explain how this unconnected
lookup works in the attached XML, then use the AI Dev Kit to create
a Spark SQL broadcast join equivalent and run it against the dev warehouse."
```

4.3 Databricks AI Dev Kit (Deployment Layer)

Role: Make the AI assistant Databricks-aware; productionize assets.

What it provides:

- **Executable actions:** 50+ tools that let the AI assistant interact directly with your Databricks workspace—execute SQL, create jobs, inspect Unity Catalog, deploy apps, trigger pipelines
- **Skills:** 20+ markdown files teaching Databricks patterns (Spark pipelines, DLT, Unity Catalog)
- **Project-scoped:** Installs into `.claude/`, `.windsurf/`, `.github/`, etc. in your repo

Install:

```
bash <(curl -sL https://raw.githubusercontent.com/databricks-solutions/ai-dev-kit/main/install.sh)
```

Critical Integration: When the AI assistant generates a converted mapping, it can immediately:

1. Execute the Spark SQL to verify compilation
2. Create a Databricks Job to test the workflow
3. Inspect Unity Catalog to confirm table creation
4. Iterate based on error logs—all without leaving the CLI.

5. Informatica Skill Tree Specification

5.1 Structure

```
.claude/skills/informatica/
├─ README.md
├─ informatica-transformations/
│   ├── source-qualifier/
│   │   ├── SKILL.md
│   │   └─ references/
│   │       └─ sql-override-examples.md
│   ├── expression/
│   │   ├── SKILL.md
│   │   └─ references/
│   │       └─ expression-editor-functions.md
│   ├── lookup/
│   │   ├── SKILL.md
│   │   └─ references/
│   │       ├── cache-mechanics.md
│   │       ├── dynamic-lookup-examples.md
│   │       └─ unconnected-lookup-patterns.md
│   ├── joiner/
│   │   ├── SKILL.md
│   │   └─ references/
│   │       ├── sorted-join-details.md
│   │       └─ blocking-behavior.md
│   ├── aggregator/
│   ├── router/
│   ├── filter/
│   ├── sorter/
│   ├── rank/
│   ├── sequence-generator/
│   └─ update-strategy/
```

- | └─ transaction-control/
- | └─ normalizer/
- | └─ stored-procedure/
- | └─ data-masking/
- | └─ custom-transformation/
- | └─ external-procedure/
- | └─ union/
- | └─ xml-source-qualifier/
- └─ informatica-workflows/
 - | └─ workflow-design/
 - | └─ session-configuration/
 - | └─ scheduling/
 - | └─ parameter-files/
 - | └─ error-handling/
- └─ informatica-repository/
 - | └─ xml-schema/
 - | └─ metadata-objects/
 - | └─ lineage-tracing/
- └─ informatica-functions/
 - | └─ date-functions/
 - | └─ string-functions/
 - | └─ numeric-functions/
 - | └─ conversion-functions/
 - | └─ conditional-functions/
 - | └─ aggregate-functions/
- └─ informatica-to-databricks-mapping/
 - | └─ transformation-mapping/
 - | └─ function-mapping/
 - | └─ workflow-orchestration-mapping/
 - | └─ performance-patterns/
 - | └─ anti-patterns/

5.2 SKILL.md Template

```

---
name: informatica-[transformation-name]
description: "Use when analyzing, explaining, or migrating Informatica PowerCenter [Transformation
↪ Name] transformations. Covers configuration properties, behavior rules, cache mechanics, and Spark/
↪ Databricks equivalents. Do NOT use for general ETL concepts, non-Informatica transformations, or
↪ BDM/DEI assets."
---

# [Transformation Name]

## Purpose
[One-sentence description from official docs]

## Configuration Properties

```

Property	Description	Default	Impact on Migration
[Prop1]	[Desc]	[Val]	[How it affects Spark rewrite]

```

## Behavior Rules
- [Rule 1: directive, not essay]
- [Rule 2: explain WHY if non-obvious]

## Spark / Databricks Equivalent
[Pattern mapping with code example]

## Edge Cases & Warnings
- [What Lakebridge might miss]
- [Manual intervention required]

## Example: Before and After
sql
-- Informatica (Expression transformation)
IIF(ISNULL(CUSTOMER_ID), 'UNKNOWN', CUSTOMER_ID)

-- Databricks (Spark SQL)
COALESCE(CUSTOMER_ID, 'UNKNOWN')

```

5.3 Reference File Rules

- Max 500 lines per `SKILL.md` body; overflow goes into `references/`
- Each reference file >500 lines needs a TOC with line hints at top
- Use directives: "Always use X", not "X is recommended"
- Lead with 5-line code snippets, not 5-paragraph explanations
- Include negative cases: when NOT to use this skill

5.4 Master Mapping Skill

The `informatica-to-databricks-mapping/` skill is the **Rosetta Stone**. It must contain:

1. **Transformation equivalence table**
2. **Function equivalence table** (Informatica → Spark SQL)
3. **Before/after code examples** for common patterns
4. **Flags for manual intervention** (unconnected lookups, blocking joins, stored procedures)
5. **Cross-references** to specific Databricks AI Dev Kit skills for implementation

6. How the AI Assistant Uses the Skills

6.1 Reading Informatica XML

The AI assistant reads exported repository XML as **plain text files** using its built-in file system
 ↔ access:

“Read the file `repository/m_Customer_Load.xml` and use the `informatica-lookup` skill to explain the unconnected lookup on the `CUSTOMER_ID` field.”

The **xml-schema skill** teaches the AI assistant the structure of PowerCenter XML:

```
- `<MAPPING>`, `<TRANSFORMATION>`, `<CONNECTOR>` nodes
- `FROMFIELD`, `TOFIELD`, `THROUGHPORT` attributes
- `<INSTANCE>`, `<WORKFLOW>`, `<SESSION>` objects
```

6.2 Interpreting Logic

With the skills loaded, the AI assistant can:

- Identify transformation types and their configurations
- Trace field-level lineage through connector chains
- Explain business logic in natural language
- Flag active vs passive transformations
- Detect blocking vs non-blocking behavior

6.3 Generating Databricks Code

The AI assistant then uses the **mapping skill** to generate equivalents:

“Based on the informatica-joiner skill, rewrite this blocking unsorted joiner as a Spark SQL LEFT JOIN with a broadcast hint. Use the AI Dev Kit to execute it on the dev cluster.”

6.4 Validation Loop

The AI Dev Kit provides the execution layer:

- Execute generated Spark SQL directly on a SQL warehouse
- Create Databricks Jobs to test workflows
- Inspect Unity Catalog to verify table creation
- Read job logs and iterate on failures

No additional infrastructure is needed—the AI assistant reads XML as text, skills provide interpretation, and the AI Dev Kit provides workspace execution.

7. Risk Register

Risk	Likelihood	Impact	Mitigation
Lakebridge fails on proprietary functions	High	Medium	AI assistant function skills + manual UDF creation
Workflow orchestration not auto-convertible	High	High	Treat as separate workstream; redesign in Databricks Workflows/Airflow
BDM/DEI assets uncovered mid-migration	Medium	High	Pre-migration inventory; engage LeapLogic/Kanerika FLIP if needed

Table 6 – continued

Risk	Likelihood	Impact	Mitigation
Team lacks Spark/Python skills	Medium	High	AI Dev Kit skills reduce barrier; plan training sprints
Data reconciliation failures	Medium	Critical	Parallel running with row-level + aggregate + semantic validation until parity is proven
AI Dev Kit / assistant hallucination	Medium	Medium	Ground with Informatica skills; validate all generated code via AI Dev Kit execution
Informatica XML schema changes	Low	Low	Version-control exported XML; document schema version used

8. Success Criteria

Metric	Target
Mappings auto-converted by Lakebridge	70%
Mappings requiring AI assistant intervention	30%
Data reconciliation pass rate	100% (row-level + aggregate)
Workflow orchestration migrated	100% (redesigned, not 1:1)
Informatica decommissioned	Domain-by-domain, 100% final
Post-migration query performance	120% of Informatica baseline (target: faster)
Unity Catalog governance coverage	100% of migrated tables

9. Next Actions

1. Confirm BDM/DEI scope; decide if commercial accelerator needed
2. Export PowerCenter repository XML + download all PDF guides
3. Install Databricks AI Dev Kit in migration project directory
4. Build Phase 0 skill tree (priority: transformations, functions, mapping)
5. Run Lakebridge assessment against exported repository
6. Establish complexity matrix and gap registry
7. Begin bulk transpilation (Phase 1)

Plan version: 1.2

Based on: Lakebridge (Databricks Labs), AI Coding Assistant, Databricks AI Dev Kit (Field Engineering), Phil Schmid's Agent Skills best practices